arXiv:2611.12935v1 [cs.AI] 6 Nov 2026

Train on prefill, monitor at decode: Using sparse autoencoders for topic monitoring

Rishikesh Mallagundla
Mercity
rishikesh@mercity.io

Pranav Patel
Mercity
pranav@mercity.io

Abstract

Topic monitoring is the task of labelling the topic of each token in a model's response during generation. Activation-based monitors that run at generation time have so far been trained to detect harmful content and are limited to two or three classes. We introduce TopicScope, a framework for training causal topic classifiers on SAE features that assign a topic to every token, so that the label can change partway through a response. We train three detector architectures on synthetic multi-topic conversations covering seven topics. Across Gemma-3 1B and 4B, monitors trained on features extracted from prefill hidden states were able to generalise to decoding. We also found that the classifiers are robust to perturbations, losing at most 5 macro-F1 points when off-topic asides are inserted, topics are removed or sections are reordered. Ablations conducted to understand the contribution of the SAE revealed that classifiers reaching baseline performance without it were several times larger. Our findings show that compact topic monitors can be trained inexpensively in prefill and deployed during generation. More broadly, our work shows where SAEs are useful beyond safety classification. Our code, dataset and checkpoints are available on **GitHub** and **Hugging Face**.

1 Introduction

Language models are frequently deployed as single-purpose assistants, for tasks such as customer support, billing or clinical triage. Most deployed assistants serve a fixed purpose and any kind of unintentional drift during response generation should be avoided to save costs and align the model to its mission. For example, a billing assistant for a telecom company should talk about invoices, payments and refunds, and avoid topics such as tax or legal advice. Even though this topic list can be ingrained in the model through a carefully designed system prompt or through fine-tuning [42], the model can still drift off topic over the course of a multi-turn conversation [39, 42].

Topic monitoring is the task of labelling the topic of each token in a model's response during generation, which downstream can enable catching the point where a response leaves its scope while it is still being

written.

Monitoring a response while it is being written is already common practice for safety. Guard models such as Llama Guard [21] and ShieldGemma [49] classify a prompt or a finished response, and streaming guardrails extend them to label the response token by token as it is generated [27, 47, 50]. A parallel line of work reads the generating model's own activations instead of its text. Linear probes on these activations screen for jailbreaks [11] and detect deception [15] and high-stakes interactions [31], and probes on sparse autoencoder features have flagged personal data token by token in production [33]. Across both lines of work, the label is almost always a safety judgement between benign and harmful, with two or three classes.

Topical guardrails, in turn, classify the prompt or the finished response [9, 39], so a change of topic is caught only after it has been written, and a single label for the response may not show where the change happened. While prior work monitors safety during generation and topic after it, no existing monitor labels the topic of a response token by token as it streams. Extending streaming safety monitors from two or three classes to many topics, read from the generating model's own features, would close this gap.

We introduce TopicScope, a framework for training topic monitors using sparse autoencoders (SAEs). TopicScope relies on the fact that, in a causal language model, the hidden state at a token is computed the same way whether that token was read in prefill or generated in decode. We therefore pass conversations annotated with topics at the token level through the backbone in prefill, train small causal detectors on the SAE features of the resulting hidden states, and then run the same detectors unchanged on the hidden states the backbone produces while it generates.

We use TopicScope to answer four research questions, each building on the one before it: **(RQ1)** Do SAE features carry a per-token topic signal? **(RQ2)** Do detectors trained in prefill transfer to decode? **(RQ3)** Do the detectors stay accurate when the response is perturbed? **(RQ4)** Is the SAE necessary?

Our core contributions are:

1. We train causal topic monitors on SAE features for Gemma-3 1B and 4B [13], using synthetic multi-topic conversations over seven topics and three detector architectures: a GRU, a ConvNeXt-style temporal convolutional network and a Transformer. We release the synthetic conversation dataset along with the code that generates it and the trained detector checkpoints on GitHub and Hugging Face.
2. We show that these monitors, trained only on teacher-written text in prefill, carry over to decode and follow the topics of the responses the backbone generates itself. They maintain their scores even when we insert off-topic text, remove a topic or reorder the sections of a response.
3. We train the same detectors on raw hidden states to test whether the SAE is needed, and find that its benefit is a monitor whose size does not grow with the backbone. The raw-state detectors reach the same performance only once they are several times larger.

The rest of the paper is organised as follows: Section 2 reviews prior work on topic monitoring and sparse autoencoders, Section 3 introduces the language models, sparse autoencoders and causal detectors we build on, Section 4 outlines TopicScope's architecture and the dataset generation pipeline, Section 5 answers the research questions and discusses the findings, and Section 6 concludes with directions for future work.

2 Background and Related Work

Topic modelling and segmentation. Topic models summarise a corpus as a set of themes. The classic approach is a bag-of-words generative model such as LDA [6], later followed by neural variants [43],

clustering of document embeddings [18] and prompted LLMs [35]. Topic segmentation splits a single document into topically coherent parts, either from lexical cohesion [20] or as a supervised sequence labelling task [25]. In dialogue, topic-shift detection asks whether the topic changes at a given utterance [28]. All of these methods work on text that has already been written. Topic models discover their themes from a whole collection, and segmentation places boundaries after the fact. TopicScope instead assigns one of a fixed set of topics to every token while the response is still being generated.

SAEs as topic models. Three recent papers link SAEs to topic modelling, and all three stay unsupervised and corpus-level like the models above. Girrbach and Akata [14] derive the SAE objective as a maximum a posteriori estimator under a continuous topic model inspired by LDA. Mechanistic Topic Models [51] and MonoTM [22] describe topics with SAE features in place of word lists, which lets a topic capture ideas that a list of top words cannot.

Topical guardrails. Most scope control in deployed systems works on the text. NeMo Guardrails [39] lets developers write programmable rails over a dialogue, including rails that keep the conversation away from given topics. Such rails depend on curated examples, and Chua et al. [9] remove that dependence by generating synthetic prompts from a description of the intended scope, then training classifiers that judge whether a user prompt is relevant to the system prompt. Fence [26] trains similar specialised small models for other application-specific checks, topic drift among them. Instead of screening the conversation, CantTalkAboutThis [42] fine-tunes the model itself to stay on topic. More recent methods move the check from the text to the model's activations. Pan [34] fit a one-class SVM on hidden states to flag conversation turns that fall out of context, and kNNGuard [1] classifies prompts by their nearest neighbours in the activation space of an off-the-shelf model. Whether they read text or activations, all of these methods make one decision per prompt, turn or response, and none labels a response while it is being generated.

Safety moderation and streaming guardrails. Moderation models such as Llama Guard [21], Shield-Gemma [49] and WildGuard [19] classify a prompt or a complete response against a harm taxonomy. Streaming guardrails move that check into generation. Stream Qwen3Guard adds a token-level head that labels output as it arrives [50]. SCM trains a token-level monitor on heuristic per-token harm labels and stops a response once enough tokens are flagged [27]. SentGuard moderates at sentence boundaries to avoid unstable token-level decisions [47], while FreoStream and StreamGuard try to anticipate harm before it is fully written [24, 45]. In timing, these systems are the closest to ours. They are separate models, often several billion parameters, that read the generated text, and their labels are safety classes with two or three values. TopicScope, by contrast, reads the generating model's own activations and separates many topics.

Activation probes as monitors. Linear probes were introduced to study what a layer encodes [2, 4], and representation engineering extended them to reading and steering high-level concepts [52]. They are now used as runtime monitors. Constitutional Classifiers++ uses a probe as a cheap first stage before a larger classifier [11]. Probes also detect strategic deception [15], high-stakes interactions [31] and reward hacking [5]. In method, this line of work is the closest to ours. Each probe, though, targets a single binary property, and most score a whole prompt or response. McKenzie et al. [31] also find that probes trained on synthetic data generalise to real interactions, which is the assumption behind our teacher-written training text.

SAE features as classifier inputs. Whether SAE features make better classifier inputs than raw activations is contested. Kantamneni et al. [23] test SAE probes under data scarcity, class imbalance, label noise and covariate shift and find no consistent gain over baselines. Smith et al. [41] report SAE

probes trailing linear probes on out-of-distribution harmful-intent detection, and AxBench reaches a similar conclusion for steering and concept detection [46]. Other results are more positive. Gallifant et al. [12] find SAE features outperforming hidden-state baselines on safety classification and transferring from Gemma 2 2B to 9B, and SAE probes trained on synthetic data detected personal data in production at Rakuten, transferring to real traffic better than other probing methods [33]. Practitioner guidance recommends against an SAE when the target concept is known in advance [16]. Topics are such a concept, and our ablation without the SAE tests this guidance on a multi-class, per-token task.

3 Preliminaries

Decoder-only language models such as GPT [36, 37] and Gemma 3 [13] are trained to predict each token from the ones before it. Given a sequence of tokens $x_1, \ldots, x_T$, such a model factorises the probability of the sequence as

$$p_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \tag{1}$$

and is trained to maximise the log-likelihood of a large text corpus under this factorisation,

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}). \tag{2}$$

To make each of these predictions, the model computes a hidden state $h_t^{(\ell)}$ at every position t in each layer ℓ of a Transformer stack [44]. These hidden states accumulate in the residual stream, which carries everything later layers use to predict the next token. Since the objective conditions each prediction on $x_{<t}$ alone, attention is masked to earlier positions, and the hidden state at position t depends only on $x_1, \ldots, x_t$.

This causal dependence holds in both ways a model processes text. During prefill, the model reads a given prompt in one forward pass, and during decode it generates one token at a time, appending each new token to the context and reusing the cached keys and values of the earlier ones.

Sparse autoencoders (SAEs) are dictionary-learning models that rewrite a language model's activations as a sparse combination of a large number of features [7, 10]. Given a residual-stream activation $h \in \mathbb{R}^d$, an SAE encodes it into a feature vector

$$f(h) = \sigma(W_{\text{enc}} h + b_{\text{enc}}) \in \mathbb{R}^m, \qquad m \gg d, \tag{3}$$

and reconstructs the activation from those features as

$$\hat{h} = W_{\text{dec}} f(h) + b_{\text{dec}}. \tag{4}$$

Training penalises the number of active features, so only a small fraction of the m features fire on a given token, and many of them respond to a single recognisable concept. The Gemma Scope 2 SAEs [17] are trained on the residual stream of every layer of Gemma 3. They use the JumpReLU activation [38],

$$\sigma(z) = z \cdot \mathbb{1}[z > \theta], \tag{5}$$

which passes a pre-activation through unchanged when it exceeds a learned per-feature threshold θ and sets it to zero otherwise.

Causal detectors. A causal detector is a sequence model that labels each position of an input sequence $z_1, \ldots, z_T$ using only that position and the ones before it,

$$\hat{y}_t = g(z_1, \ldots, z_t). \tag{6}$$

Such a detector can emit the label for a token as soon as the token is produced, which is what a monitor running alongside decode requires, since the tokens after it do not exist yet. Recurrent, convolutional and attention-based models meet this constraint in different ways, and each makes a different assumption about how much of the past a label needs.

A gated recurrent unit (GRU) [8] keeps a hidden state s_t that it updates once per token from the previous state and the current input,

$$s_t = \mathrm{GRU}(s_{t-1}, z_t), \tag{7}$$

with gates that decide how much of the old state to keep and how much to overwrite. Because s_t depends only on s_{t-1} and z_t, the GRU is causal by construction and needs no mask or padding. The same recurrence means each new token costs one state update, no matter how long the sequence has grown. The whole history, however, has to fit in a fixed-size state, so a topic set many tokens earlier survives only as long as the gates keep it.

A temporal convolutional network (TCN) [3] labels each token from a window of recent inputs instead of a running state. A one-dimensional convolution becomes causal when its input is padded on the left only, so the output at t covers positions $t-k+1, \ldots, t$ for a kernel of size k. Stacking such layers with dilations that double at each layer widens this window exponentially with depth. ConvNeXt blocks [29] can serve as these layers. Each block pairs a depthwise convolution with a per-token MLP and a residual connection, and since the depthwise convolution is the only part that mixes positions, padding it on the left is enough to make the whole block causal.

A Transformer encoder [44] lets the representation at position t attend directly to every earlier position. Applying the same lower-triangular mask that a decoder-only language model uses blocks attention to later positions, and this mask alone makes the encoder causal. Where the GRU compresses the past into a state and the TCN restricts it to a fixed window, the Transformer keeps the full history available at every token, at a cost that grows with the length of the sequence.

4 Methodology

4.1 Overview of TopicScope

TopicScope trains a topic monitor in prefill and runs it in decode (Figure 1). The pipeline has four steps:

1. **Data.** A teacher model writes a corpus $\mathscr{D}$ of $|\mathscr{D}| = 10{,}000$ conversations, each divided into topic-labelled spans in both the prompt and the response.
2. **Prefill.** Each conversation passes through a backbone language model $\mathscr{M}$ in a single forward pass, and we read the residual stream at its last N layers.
3. **Features.** At each of these layers, a pretrained SAE $\mathscr{S}$ encodes the residual stream, and a selection step reduces the encoded features to a fixed-width input z_t for every token.
4. **Detection.** A small causal detector learns to predict the topic of every token from z_t. At deployment, $\mathscr{M}$ writes its own response, and the same detector, unchanged, labels each token as it is generated.

Only the detector is trained; the LLM and the SAEs stay frozen.

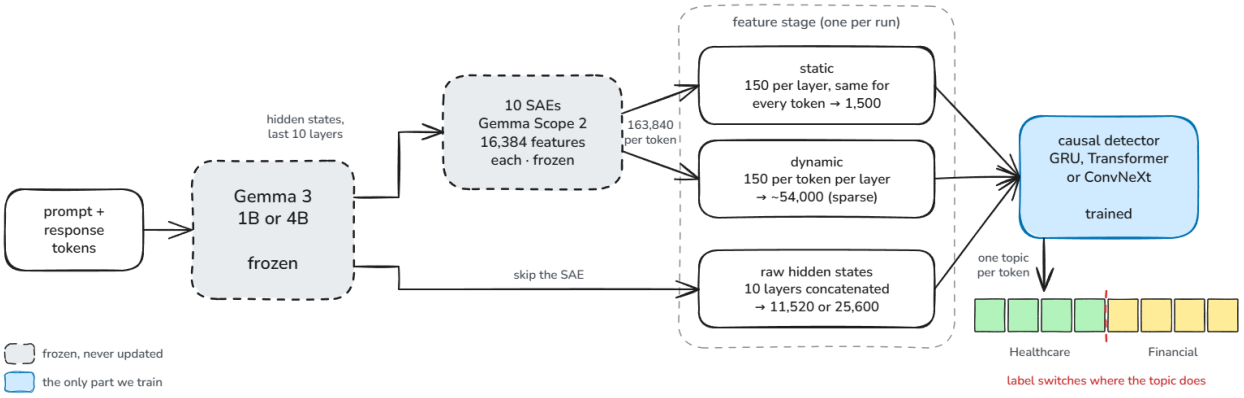


Figure 1: The TopicScope pipeline. Gemma 3 and its SAEs are frozen, a feature stage reduces the SAE output at ten layers to the detector's input, and only the causal detector is trained. The detector emits one topic per token, so its label can switch partway through a response.

4.2 Task formulation

Given a conversation tokenised as $x_1, \ldots, x_T$, made of $\boldsymbol{T_p}$ prompt tokens followed by $\boldsymbol{T_r}$ response tokens under the backbone's chat template, topic monitoring assigns each token a label $y_t \in \mathcal{Y}$ from a set of topics $\mathcal{Y}$. We use $|\mathcal{Y}| = 7$ business topics (Section 4.3). A single response may cover several of these in sequence, so the label can change partway through, and the monitor has to place that change at the token where it happens. As in Section 3, the monitor is causal and predicts

$$\hat{y}_t = g(z_1, \ldots, z_t), \tag{8}$$

where z_t is the feature vector read from the backbone at token t. Tokens in both T_p and T_r carry labels when they fall inside a topic span.

The monitor is trained in prefill, where a finished conversation passes through the backbone in one forward pass and every token has a label, and deployed in decode, where the backbone writes one token at a time and no labels exist.

4.3 Dataset

The corpus is generated with Simula [32], our synthetic data generation library, using GPT-5.6 Luna as both generator and critic. For each sample, Simula draws a mix from a custom taxonomy built on the seven topics (Enterprise documents, General news & content, Customer service, Legal, Financial, HR & people operations, and Healthcare), writes a short brief describing the record to create, and generates the record from that brief. A critic then checks every record against its brief and the labelling rules, and rejected records are refined or discarded.

Each record is a single prompt–response pair, in which a short user prompt asks about one or more topics and an assistant response answers them in order. The topics and their order are fixed by the sampled mix before the record is written, and the generator marks each topic's span in both the prompt and the response, which gives every token its label (Figure 2). Across the corpus, most pairs cover more than one topic, the seven topics are balanced, and topic sequences are sampled so that every ordered pair of topics can occur (Table 1 and Figure 3).

Generation also varies how each topic change is written, so that a monitor cannot find a boundary from surface cues alone. Every multi-topic pair uses one pivot style for all its boundaries: an explicit

Prompt

What are the main developments in recent climate and technology news? Also help me resolve a duplicate charge on my account. Finally, turn the relevant operational lessons into a concise internal memo.

Response (excerpt)

Recent climate coverage has focused on a combination of record-setting heat, rapid investment in clean energy, and the continuing difficulty of reducing emissions from heavy industry. [...] Across both areas, the news points to fast-moving change accompanied by practical questions about infrastructure, accountability, and public policy. A duplicate charge is usually best handled by first comparing the transaction details rather than assuming that two entries represent the same payment. [...] Review the account afterward to make sure the order status and any refund are both recorded accurately. The operational lesson is to make payment incidents visible early without treating every temporary authorization as a confirmed error. [...]

General news & content Customer service Enterprise documents

Figure 2: A test record that covers three topics. Each highlight marks a topic's labelled span in the prompt and the response, and the response is excerpted around its two topic changes.

Size		Topics	
Records	10,000	Mean topics per record	2.65
Train / validation / test	7,000 / 1,500 / 1,500	Records with 1 topic	2,067
Total tokens	8.85M	Records with 2 topics	2,128
Labelled response tokens	8.2M	Records with 3 topics	3,092
Labelled prompt tokens	515K	Records with 4 topics	2,713
Mean prompt length	44 words	Records containing each topic	3,687 to 3,842
Mean response length	675 words		
Pivot style (share of multi-topic records)		Explicit 40%, light 35%, unmarked 25%	

Table 1: The synthetic topic-shift corpus.

connective such as "On a separate note", a light or neutral connective, or an unmarked pivot that starts the new topic directly or reaches it through a person, company or object the two passages share.

4.4 Feature extraction

We use Gemma 3 1B and 4B as backbones [13], and their corresponding Gemma Scope 2 SAEs at the last $N = 10$ layers (16–25 for 1B and 24–33 for 4B) to extract features from the residual stream.

For efficiency, we keep only $K = 150$ features per layer and concatenate them across the N layers into the detector's input z_t (Figure 4), using one of the following methods to select them:

- **Static selection** fixes one set of K features per layer, fitted on the training split and shared by every token, using one of three rules for ranking features:
  - **Summation** keeps the K features with the largest summed activation over the training tokens.
  - **F-statistic** keeps the K features whose mean activation differs most between topics relative to their

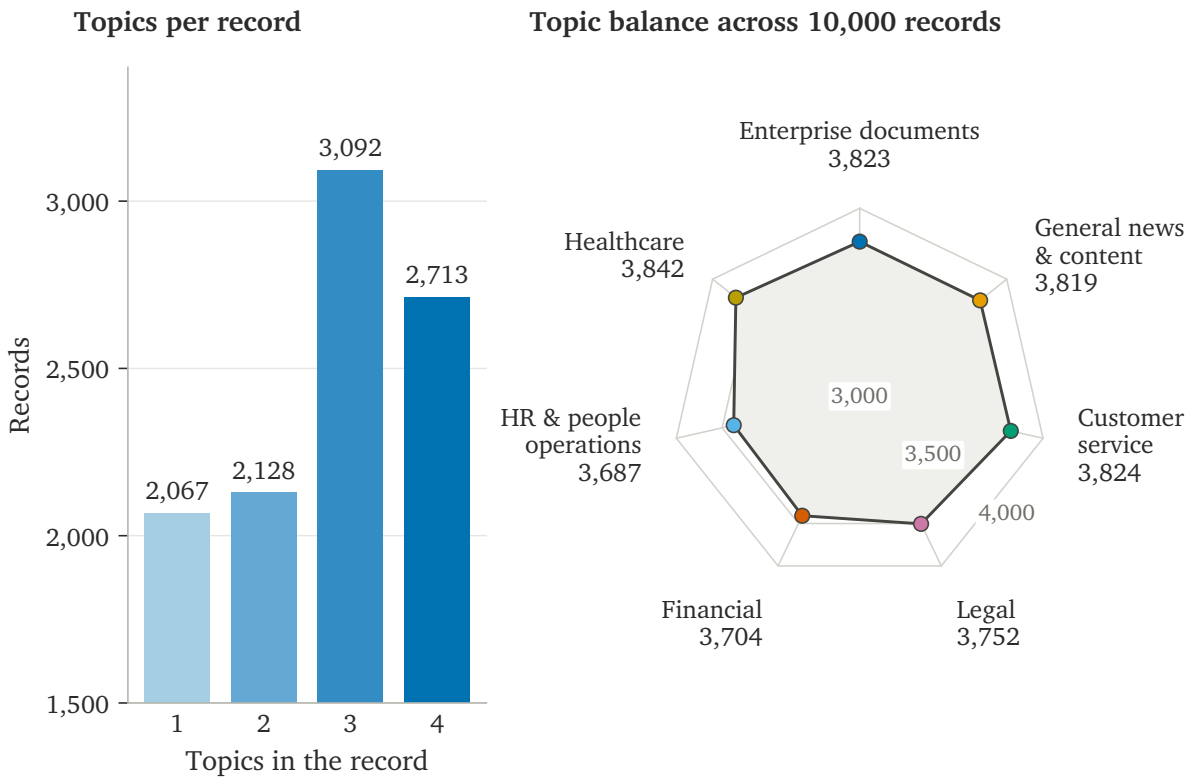


Figure 3: Composition of the corpus: records by number of topics (left) and records containing each topic (right). The bar axis starts at 1,500 records and the spider chart's centre is 3,000 records, not zero.

spread within a topic.

  - **Group lasso** fits a linear probe from all m features to the seven topics with a sparse group-lasso penalty [48].
- **Dynamic selection** keeps the top-K features of each token at every layer, each written into a fixed column for that feature, so the set of active columns changes from token to token.
- **NoSAE**, our baseline, bypasses the SAE altogether and sends the hidden states of the N layers to the detector directly.

4.5 Detectors and training

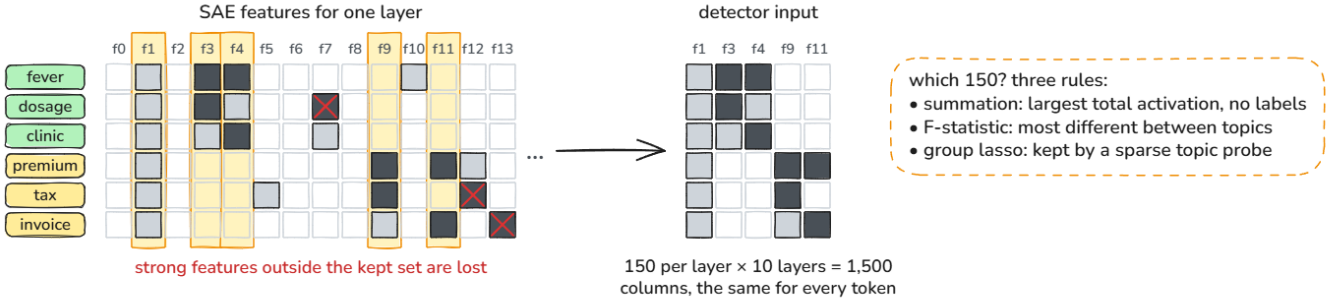
We train the three causal detectors from Section 3, (1) a **GRU**, (2) a **ConvNeXt**-style temporal convolutional network and (3) a causal **Transformer** encoder, on every feature input with token-level cross-entropy over the labelled tokens, using AdamW [30] for 15 epochs, and keep the checkpoint with the best validation macro-F1. Architectures, parameter counts and training hyperparameters are listed in Section C.

5 Results

Every score below comes from the 1,500 held-out test records and is a percentage. Where tokens carry labels, we report **macro-F1**, the token-level F1 averaged equally over the seven topics, and **overlap**, the intersection-over-union of predicted and true tokens per topic averaged over topics. Overlap is the stricter of the two, because it counts each correct token once where F1 counts it twice.

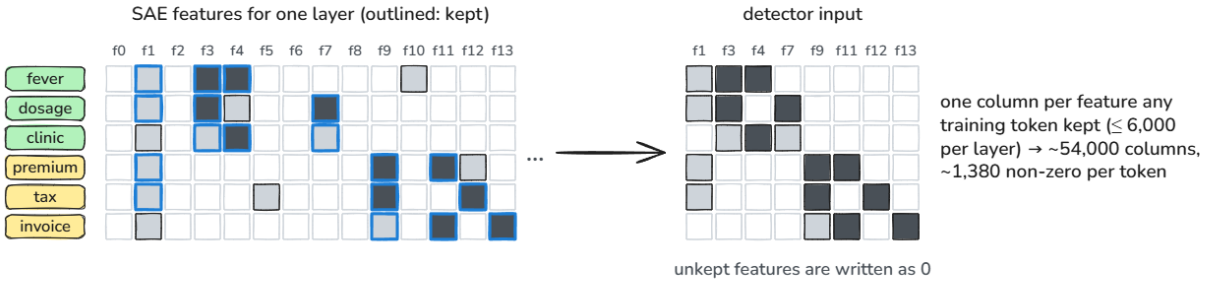
Static: pick the columns once

A rule fitted on the training split keeps 150 of 16,384 features per layer. Every token gets the same columns.



Dynamic: pick again for every token

Each token keeps its own 150 strongest features per layer. Each kept value goes in the column that belongs to its feature.



NoSAE: skip the SAE

Each layer's hidden state goes to the detector as it is, and the ten layers are concatenated.

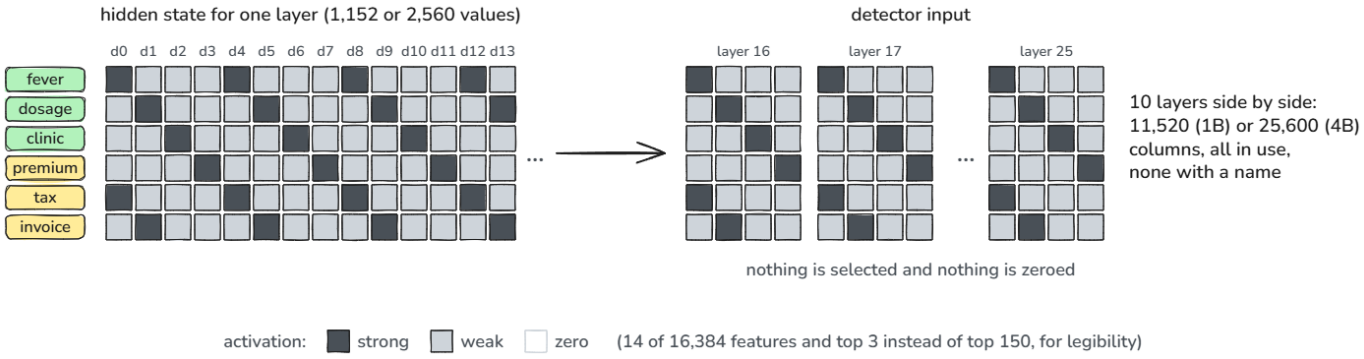


Figure 4: Static selection keeps the same columns for every token, so a token whose strongest features fall outside them loses those features. Dynamic selection keeps each token's own strongest features and writes each into the column that belongs to it. NoSAE skips the SAE and passes the hidden states directly.

5.1 RQ1: Do SAE features carry a per-token topic signal?

We train the three detectors on every feature input for both backbones and score them in prefill (Table 2), and report per-topic scores, the effect of supervising prompt tokens, and layer knockout and attribution in Sections A and B.

		Macro-F1 (%)			Overlap (%)		
	Input	GRU	ConvNeXt	Transformer	GRU	ConvNeXt	Transformer
Gemma 3 1B	Summation	94.5	96.2	94.3	89.6	92.6	89.3
	F-statistic	96.4	96.2	95.7	93.0	92.8	91.8
	Group lasso	**96.8**	96.3	95.5	**93.8**	92.9	91.5
	Dynamic	96.1	**97.3**	94.0	92.5	**94.8**	88.7
	NoSAE	96.5	96.7	**95.9**	93.3	93.6	**92.2**
Gemma 3 4B	Summation	87.4	92.4	91.8	77.7	86.0	85.0
	F-statistic	**96.4**	95.5	**95.0**	**93.1**	91.5	**90.5**
	Group lasso	96.3	95.5	94.9	92.9	91.4	90.3
	Dynamic	92.4	97.1	94.0	86.0	94.5	88.7
	NoSAE	96.3	**97.5**	94.9	92.9	**95.1**	90.4

Table 2: Test macro-F1 and overlap for every input. The NoSAE rows are discussed in Section 5.4.

On Gemma 3 1B, even the unsupervised summation rule reaches 94.3–96.2 macro-F1 from 1,500 of the 163,840 available features, and selecting features per token (dynamic selection) adds at most 1.1 points. This parity does not hold on Gemma 3 4B, where every summation detector scores below its Gemma 3 1B counterpart and GRU drops to 87.4.

Holding the budget at $K = 150$ features per layer and changing only the selection rule removes this drop. Both label-based rules bring every Gemma 3 4B detector to within a point of its Gemma 3 1B score and raise the macro-F1 of the Gemma 3 4B GRU from 87.4 to 96.4. The two supervised rules are within 0.4 points of each other in every row, so the F-statistic, a single deterministic pass with no hyperparameters, is the cheaper choice.

Among the detectors, ConvNeXt is the strongest in six of the ten rows, and GRU depends most on the input, spanning 87.4 to 96.8.

5.2 RQ2: Do detectors trained in prefill transfer to decode?

The detectors from RQ1 then label, without retraining, responses that the backbone generates itself for 1,000 test prompts with the topic tags removed. These responses have no per-token labels, so each is scored against the topics its prompt requested:

- **Purity:** the share of response tokens labelled with a requested topic.
- **Coverage:** the share of requested topics that each receive at least a tenth of the tokens.
- **Full coverage:** the share of responses in which every requested topic clears that bar.

These measures test consistency with the request. A low score can come from the detector mislabelling the response or from a response that strayed from the request. To separate the two, the same detectors also read the teacher's responses to the same prompts. Those responses cover every requested topic, so their scores serve as the reference.

The detectors transfer. Gemma's responses differ from the teacher's, averaging about 1,240 tokens

		Purity (%)			Full coverage (%)		
	Input	GRU	ConvNeXt	Transformer	GRU	ConvNeXt	Transformer
Gemma 3 1B	Summation	91.1	89.9	94.5	68.1	64.3	66.9
	F-statistic	93.2	92.2	96.5	62.7	73.0	69.3
	Group lasso	**96.6**	93.8	**97.7**	69.7	75.6	67.8
	Dynamic	94.5	94.4	94.1	70.4	77.1	**73.1**
	NoSAE	95.8	**95.1**	94.7	**77.1**	**78.5**	63.7
Gemma 3 4B	Summation	69.4	75.8	90.2	49.5	33.7	69.1
	F-statistic	95.3	90.4	**96.3**	82.2	81.0	76.7
	Group lasso	**96.7**	92.5	94.0	84.1	82.1	69.9
	Dynamic	94.2	94.5	94.2	77.2	80.3	73.0
	NoSAE	95.3	**95.1**	88.3	**85.8**	**82.6**	**80.8**

Table 3: Decode scores on the backbone's own responses.

against 820, yet with dynamic or label-selected features 90–98% of its tokens receive a requested topic, at both sizes and for all three detectors (Table 3). The exception is summation on Gemma 3 4B, the weakest input in prefill, where the gap widens. The Gemma 3 4B GRU trailed dynamic selection by 5 macro-F1 points on teacher text but trails it by 25 purity points on Gemma's. Label-based selection corrects this, raising GRU's purity from 69 to 95 and ConvNeXt's full coverage from 34 to 81.

Full coverage is the weakest decode score (Table 4). On the teacher's text the same detectors find every topic the text contains, reaching 95–99 full coverage. On Gemma's text it falls by about 30 points on Gemma 3 1B and further on Gemma 3 4B, and it drops as prompts request more topics, from near 100 on single-topic prompts to 34–62 on four-topic prompts. Raising the generation limit from 1,500 to 8,000 tokens seems to not change the scores, as Gemma's responses end well before either limit. Most of the coverage gap therefore comes from Gemma, which answers fewer of the topics in a crowded prompt.

		Purity (%)		Full coverage (%)	
		Teacher	Gemma	Teacher	Gemma
Gemma 3 1B	GRU	99.1	91.1	98.6	**68.1**
	ConvNeXt	**99.6**	89.9	**98.7**	64.3
	Transformer	99.1	**94.5**	96.1	66.9
Gemma 3 4B	GRU	96.3	69.4	94.6	49.5
	ConvNeXt	98.1	75.8	**96.1**	33.7
	Transformer	**98.7**	**90.2**	96.0	**69.1**

Table 4: Summation detectors on the teacher's responses and on the backbone's own.

The measures do not test where a detector places a topic switch, only which topics it uses. Checking boundary placement in decode would need gold labels for generated text, which we leave to future work.

5.3 RQ3: Do the detectors stay accurate when the response is perturbed?

The same detectors label held-out responses under four perturbations, each scored against its unedited original and passed through the backbone from scratch:

1. **Insertion** of short off-topic asides (jokes or small talk) at topic boundaries, excluded from scoring
2. **Removal** of one requested topic from the response, leaving it in the prompt
3. **Reordering** of the topic passages so none follows the prompt's order
4. **Substitution** of a passage with another on the same topic from a different record, as a control

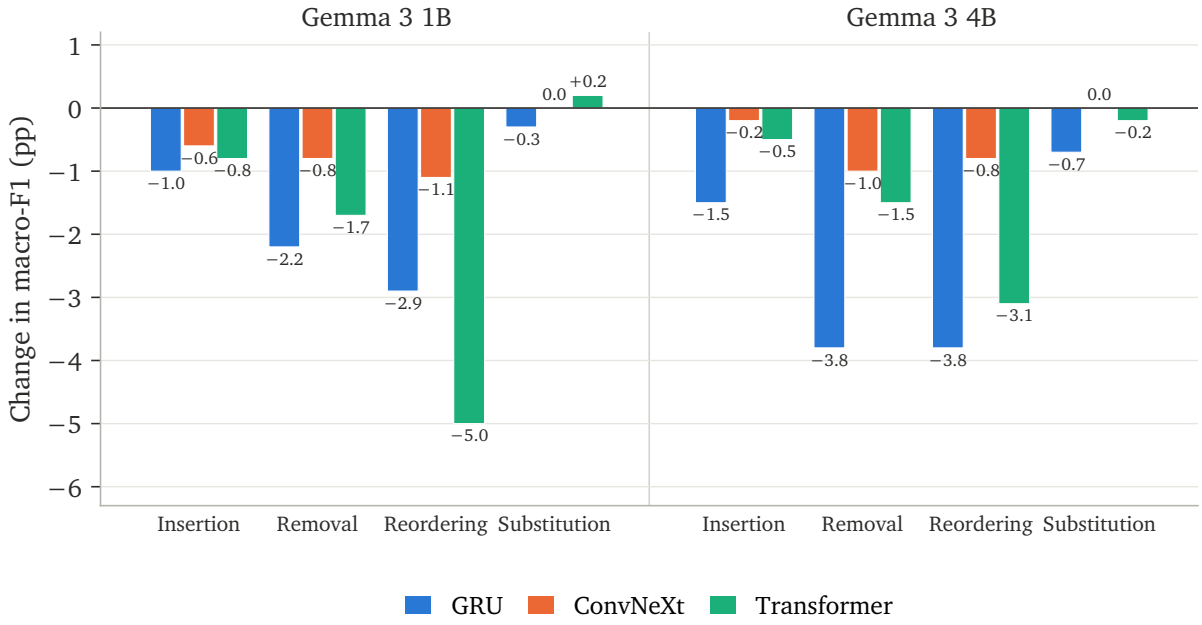


Figure 5: Change in macro-F1 (percentage points) under each perturbation, dynamic features.

No perturbation reduces macro-F1 by more than 5 points (Figure 5), and the substitution control leaves scores unchanged, which indicates that the detectors label each passage by its own content. ConvNeXt loses the least under every perturbation, never more than 1.1 points. Reordering affects Transformer the most (−5.0 on Gemma 3 1B). It attends to the whole prompt, and every training response answers the topics in the order the prompt asks them, whereas ConvNeXt's 379-token window mostly excludes the prompt.

Insertion across inputs. The loss on real tokens stays within 3.6 macro-F1 points for each of the three inputs tested, F-statistic, group lasso and dynamic (Figure 6). Most of this loss is precision on General news & content, the topic closest in content to the inserted asides. For the Gemma 3 1B F-statistic GRU, General news precision falls from 84 to 61 while the other topics move by 1 or 2 points.

	Gemma 3 1B			Gemma 3 4B		
Input	GRU	ConvNeXt	Transformer	GRU	ConvNeXt	Transformer
F-statistic	68.6	79.4	81.1	83.5	58.6	**89.6**
Group lasso	**91.0**	**91.7**	**86.0**	80.3	**80.6**	82.4
Dynamic	90.7	53.7	82.2	**87.0**	50.9	84.7

Table 5: AUROC (%) of detector confidence separating real tokens from inserted ones (50 = chance).

Because the label set has no "none" class, inserted asides still receive a topic, mostly General news & content, and a detector can flag them only if its confidence drops. For GRU and Transformer it usually does (AUROC 80–91, Table 5), but the dynamic ConvNeXt scores 54 and 51, giving inserted asides about

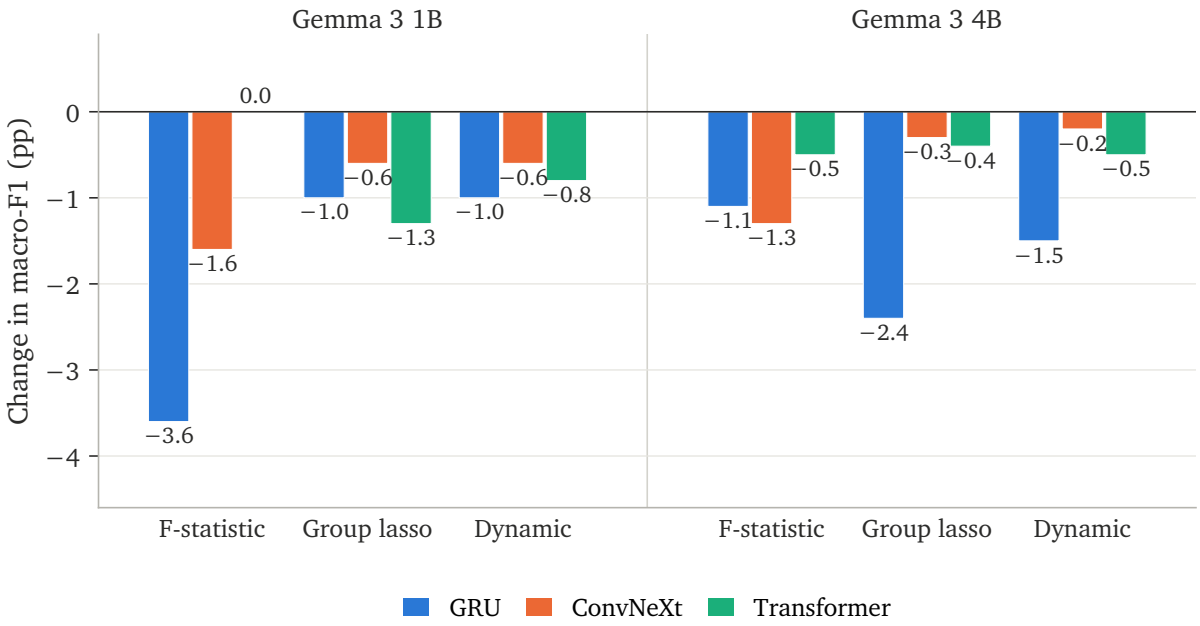


Figure 6: Change in macro-F1 (percentage points) on real tokens after insertion.

the same confidence as real tokens. Its confidence therefore cannot flag inserted text under dynamic features, and group lasso is the only input on which all three detectors, ConvNeXt included, reach 80 at both sizes.

Removal. Averaged over topics, each of these inputs assigns the removed topic to 1 to 4% of response tokens (Figure 7). The rate depends on the topic, from 7 to 8% for Enterprise documents, the topic hardest to separate in RQ1 and two to three times more affected than any other, down to 1 to 2% for Healthcare. ConvNeXt assigns the removed topic least often, but does so with 90 to 100% confidence, consistent with its uniform confidence under insertion.

Reordering. Under the label-based static rules, Transformer loses 3 to 4 macro-F1 points and ConvNeXt about 1, the same pattern as with dynamic features. Enterprise documents and HR are again the most affected topics, and Healthcare and General news the least.

5.4 RQ4: Is the SAE necessary?

We repeat RQ1 and RQ2 with the SAEs ablated, training the same detectors on the raw hidden states of the N layers (NoSAE in Tables 2 and 3), to see whether the SAE contributes to accuracy, size, or both.

Input	Width	GRU	ConvNeXt	Transformer
Static SAE (Summation, F-statistic, Group lasso)	1,500	2.1M	3.6M	1.4M
NoSAE, Gemma 3 1B	11,520	9.8M	6.1M	4.0M
NoSAE, Gemma 3 4B	25,600	20.7M	9.7M	7.6M
Dynamic	∼54,000	42.5M	17.0M	14.9M

Table 6: Detector parameters for each input. Only the input projection changes.

The SAE is not necessary for accuracy. Raw hidden states are the best input for Transformer on Gemma

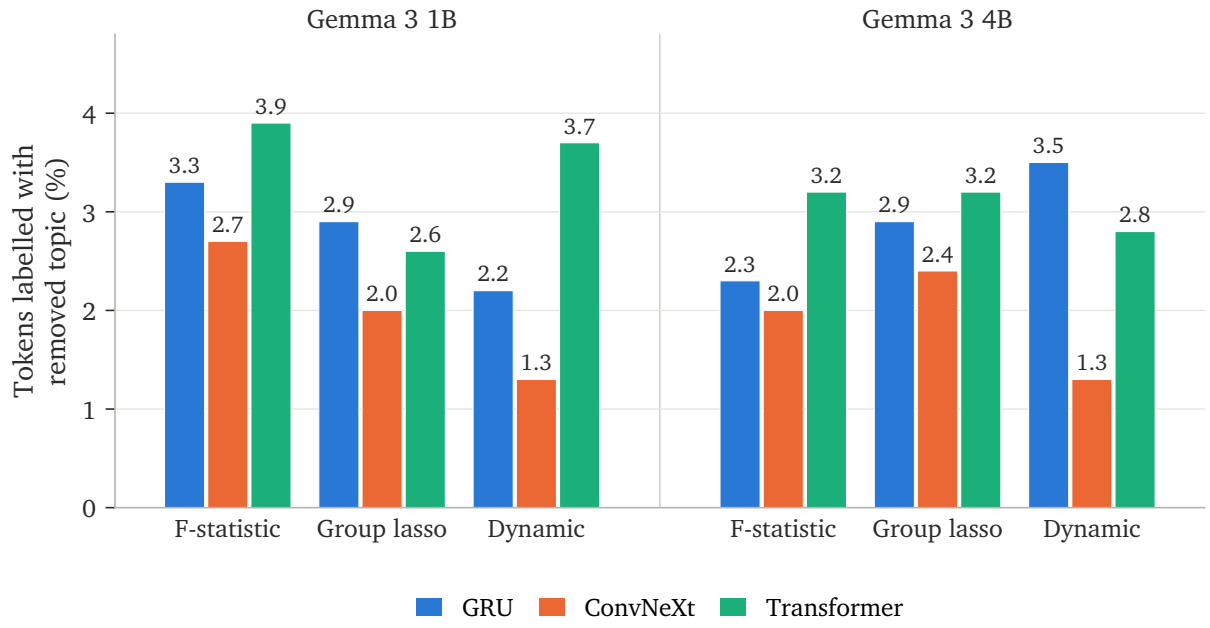


Figure 7: Share of response tokens (%) labelled with the removed topic.

3 1B and ConvNeXt on Gemma 3 4B, and within 0.3 points of the best in three of the remaining four cases. The exception is dynamic ConvNeXt on Gemma 3 1B, which leads NoSAE by 0.6 (97.3 against 96.7). The NoSAE detectors also transfer to decode at least as well, with 95–96 purity for GRU and ConvNeXt and the highest full coverage on Gemma 3 4B.

A static SAE detector, by contrast, has only 1.4M–3.6M parameters at either backbone size (Table 6), because its input stays 1,500 wide, while an NoSAE detector's input grows with the model and reaches 7.6M–20.7M on Gemma 3 4B. With label-based selection, this smaller input matches NoSAE for GRU (96.4 against 96.3) and Transformer (95.0 against 94.9) on Gemma 3 4B. ConvNeXt is the exception, where NoSAE leads 97.5 to 95.5. At inference the static stage needs only the selected encoder rows, 1.73M parameters on Gemma 3 1B and 3.84M on Gemma 3 4B, against 378M and 839M for the ten SAEs loaded whole, and every input column is a named feature that can be inspected.

6 Discussion and Conclusion

We introduce TopicScope, a framework for training topic monitors on the SAE features of a generating language model, which assign a topic to every token of a response while it is being written. Because the hidden state at a token is computed the same way in prefill and in decode, monitors trained only on teacher-written conversations in prefill transfer without retraining to the responses Gemma 3 1B and 4B generate themselves, labelling 90–98% of tokens with a requested topic when the features are selected per token or by topic labels. These monitors remain accurate when the response is perturbed, losing at most 5 macro-F1 points when off-topic asides are inserted, a topic is removed or the sections are reordered. Training the same detectors on raw hidden states shows that the SAE is not required for accuracy, but it keeps the monitor small: a static SAE detector has 1.4M–3.6M parameters at either backbone size, whereas a detector on raw hidden states grows with the backbone and reaches 7.6M–20.7M parameters on Gemma 3 4B. Every input column of the SAE detector is also a named feature, which makes the monitor open to inspection in a way raw hidden states are not.

Several directions follow from this work. Decode evaluation would be more complete with gold labels

for generated text, which would test where a monitor places each topic switch and not only which topics it assigns. A dedicated off-topic class would let the monitor flag inserted asides directly, without relying on a drop in its confidence. The perturbation analysis could be extended to detectors on raw hidden states, completing the comparison with the SAE inputs. Training on multi-turn conversations collected from deployed assistants would test the monitors on the gradual drift that motivates this work. Larger backbones, other model families and other SAE suites would show whether the fixed monitor size holds as the backbone grows further.

References

[1] Mahmoud Abdelfattah, Hamid Nasiri, and Peter Garraghan. kNNGuard: Turning LLM Hidden Activations into a Training-Free Configurable Guardrail. *arXiv preprint arXiv:2607.02072*, 2026. URL https://arxiv.org/abs/2607.02072. 3

[2] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016. URL https://arxiv.org/abs/1610.01644. 3

[3] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. *arXiv preprint arXiv:1803.01271*, 2018. URL https://arxiv.org/abs/1803.01271. 5

[4] Yonatan Belinkov. Probing Classifiers: Promises, Shortcomings, and Advances. *arXiv preprint arXiv:2102.12452*, 2021. URL https://arxiv.org/abs/2102.12452. 3

[5] L. Bergen, U. Bhalla, A. Lee, et al. Models know when they're reward hacking, and we can catch them at scale. Goodfire Research (Sep 17, 2026), 2026. URL https://www.goodfire.com/research/reward-hacking-activation-monitors. 3

[6] D. M. Blei, A. Y. Ng, and M. I. Jordan. Latent Dirichlet Allocation. JMLR 3:993-1022, 2003. URL https://www.jmlr.org/papers/v3/blei03a.html. 2

[7] T. Bricken, A. Templeton, J. Batson, et al. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. Transformer Circuits Thread, 2023. URL https://transformer-circuits.pub/2023/monosemantic-features. 4

[8] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *arXiv preprint arXiv:1406.1078*, 2014. URL https://arxiv.org/abs/1406.1078. 5

[9] Gabriel Chua, Shing Yee Chan, and Shaun Khoo. A Flexible Large Language Models Guardrail Development Methodology Applied to Off-Topic Prompt Detection. *arXiv preprint arXiv:2411.12946*, 2024. URL https://arxiv.org/abs/2411.12946. 2, 3

[10] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse Autoencoders Find Highly Interpretable Features in Language Models. *arXiv preprint arXiv:2309.08600*, 2023. URL https://arxiv.org/abs/2309.08600. 4

[11] Hoagy Cunningham, Jerry Wei, Zihan Wang, Andrew Persic, Alwin Peng, Jordan Abderrachid, Raj Agarwal, Bobby Chen, Austin Cohen, Andy Dau, Alek Dimitriev, Rob Gilson, Logan Howard, Yijin Hua, Jared Kaplan, et al. Constitutional Classifiers++: Efficient Production-Grade Defenses against Universal Jailbreaks. *arXiv preprint arXiv:2601.04603*, 2026. URL https://arxiv.org/abs/2601.04603. 2, 3

[12] Jack Gallifant, Shan Chen, Kuleen Sasse, Hugo Aerts, Thomas Hartvigsen, and Danielle S. Bitterman. Sparse Autoencoder Features for Classifications and Transferability. *arXiv preprint arXiv:2502.11367*, 2025. URL https://arxiv.org/abs/2502.11367. 4

[13] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, et al. Gemma 3 Technical Report. *arXiv preprint arXiv:2503.19786*, 2025. URL https://arxiv.org/abs/2503.19786. 2, 4, 7

[14] Leander Girrbach and Zeynep Akata. Sparse Autoencoders are Topic Models. *arXiv preprint arXiv:2511.16309*, 2025. URL https://arxiv.org/abs/2511.16309. 3

[15] Nicholas Goldowsky-Dill, Bilal Chughtai, Stefan Heimersheim, and Marius Hobbhahn. Detecting Strategic Deception Using Linear Probes. *arXiv preprint arXiv:2502.03407*, 2025. URL https://arxiv.org/abs/2502.03407. 2, 3

[16] Goodfire. A practical guide to sparse autoencoders (SAEs). Goodfire blog (Sep 28, 2026), 2026. URL https://www.goodfire.com/blog/a-practical-guide-to-saes. 4

[17] Google DeepMind. Gemma Scope 2: helping the AI safety community deepen understanding of complex language model behavior. Google DeepMind blog (Dec 19, 2025), 2025. URL https://deepmind.google/blog/gemma-scope-2-helping-the-ai-safety-community-deepen-understanding-of-complex-language-model-behavior. 4

[18] Maarten Grootendorst. BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv preprint arXiv:2203.05794*, 2022. URL https://arxiv.org/abs/2203.05794. 3

[19] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs. *arXiv preprint arXiv:2406.18495*, 2024. URL https://arxiv.org/abs/2406.18495. 3

[20] M. A. Hearst. TextTiling: Segmenting Text into Multi-paragraph Subtopic Passages. Computational Linguistics 23(1):33-64, 1997. URL https://aclanthology.org/J97-1003/. 3

[21] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674*, 2023. URL https://arxiv.org/abs/2312.06674. 2, 3

[22] Una Joh and Bei Yu. Beyond Top Words: MonoTM for Topic Modeling with Interpretable Monosemantic Features. *arXiv preprint arXiv:2609.09575*, 2026. URL https://arxiv.org/abs/2609.09575. 3

[23] Subhash Kantamneni, Joshua Engels, Senthooran Rajamanoharan, Max Tegmark, and Neel Nanda. Are Sparse Autoencoders Useful? A Case Study in Sparse Probing. *arXiv preprint arXiv:2502.16681*, 2025. URL https://arxiv.org/abs/2502.16681. 3

[24] Pride Kavumba, Koki Wataoka, Huy H. Nguyen, Jiaxuan Li, and Masaya Ohagi. Predict, Don't React: Value-Based Safety Forecasting for LLM Streaming. *arXiv preprint arXiv:2604.03962*, 2026. URL https://arxiv.org/abs/2604.03962. 3

[25] Omri Koshorek, Adir Cohen, Noam Mor, Michael Rotman, and Jonathan Berant. Text Segmentation as a Supervised Learning Task. *arXiv preprint arXiv:1803.09337*, 2018. URL https://arxiv.org/abs/1803.09337. 3

[26] Kumud Lakara, Ruibo Shi, and Fran Silavong. Fence: Specialized SLM Guardrails for LLM Applications. *arXiv preprint arXiv:2607.18268*, 2026. URL https://arxiv.org/abs/2607.18268. 3

[27] Yang Li, Qiang Sheng, Yehan Yang, Xueyao Zhang, and Juan Cao. From Judgment to Interference: Early Stopping LLM Harmful Outputs via Streaming Content Monitoring. *arXiv preprint arXiv:2506.09996*, 2025. URL https://arxiv.org/abs/2506.09996. 2, 3

[28] Jiangyi Lin, Yaxin Fan, Xiaomin Chu, Peifeng Li, and Qiaoming Zhu. Multi-Granularity Prompts for Topic Shift Detection in Dialogue. *arXiv preprint arXiv:2305.14006*, 2023. URL https://arxiv.org/abs/2305.14006. 3

[29] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. *arXiv preprint arXiv:2201.03545*, 2022. URL https://arxiv.org/abs/2201.03545. 5

[30] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. *arXiv preprint arXiv:1711.05101*, 2017. URL https://arxiv.org/abs/1711.05101. 8

[31] Alex McKenzie, Urja Pawar, Phil Blandfort, William Bankes, David Krueger, Ekdeep Singh Lubana, and Dmitrii Krasheninnikov. Detecting High-Stakes Interactions with Activation Probes. *arXiv preprint arXiv:2506.10805*, 2025. URL https://arxiv.org/abs/2506.10805. 2, 3

[32] Mercity. Building Simula: Our Synthetic Data Generation Library. mercity.ai, 2026. URL https://www.mercity.ai/research/building-simula-our-synthetic-data-generation-library/. 6

[33] N. Nguyen, M. Deng, D. Gala, M. Byun, K. Naruse, F. G. Virgo, D. Hazra, L. Gorton, D. Balsam, T. McGrath, M. Takei, and Y. Kaji. Deploying Interpretability to Production with Rakuten: SAE Probes for PII Detection. Goodfire Research (Oct 28, 2025), 2025. URL https://www.goodfire.com/research/rakuten-sae-probes-for-pii-detection. 2, 4

[34] Jonathan Pan. Conversational Context Classification: A Representation Engineering Approach. *arXiv preprint arXiv:2601.12286*, 2026. URL https://arxiv.org/abs/2601.12286. 3

[35] Chau Minh Pham, Alexander Hoyle, Simeng Sun, Philip Resnik, and Mohit Iyyer. TopicGPT: A Prompt-based Topic Modeling Framework. *arXiv preprint arXiv:2311.01449*, 2023. URL https://arxiv.org/abs/2311.01449. 3

[36] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. Improving Language Understanding by Generative Pre-Training. OpenAI technical report, 2018. URL https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf. 4

[37] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language Models are Unsupervised Multitask Learners. OpenAI technical report, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf. 4

[38] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders. *arXiv preprint arXiv:2407.14435*, 2024. URL https://arxiv.org/abs/2407.14435. 4

[39] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. *arXiv preprint arXiv:2310.10501*, 2023. URL https://arxiv.org/abs/2310.10501. 1, 2, 3

[40] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning Important Features Through Propagating Activation Differences. *arXiv preprint arXiv:1704.02685*, 2017. URL https://arxiv.org/abs/1704.02685. 19

[41] L. Smith, S. Rajamanoharan, A. Conmy, C. McDougall, T. Lieberum, J. Kramár, R. Shah, and N. Nanda. Negative Results for SAEs On Downstream Tasks and Deprioritising SAE Research (GDM Mech Interp Team Progress Update #2). AI Alignment Forum (Mar 26, 2025), 2025. URL https://www.alignmentforum.org/posts/4uXCAJNuPKtKBsi28/. 3

[42] Makesh Narsimhan Sreedhar, Traian Rebedea, Shaona Ghosh, Jiaqi Zeng, and Christopher Parisien. CantTalkAboutThis: Aligning Language Models to Stay on Topic in Dialogues. *arXiv preprint arXiv:2404.03820*, 2024. URL https://arxiv.org/abs/2404.03820. 1, 3

[43] Akash Srivastava and Charles Sutton. Autoencoding Variational Inference For Topic Models. *arXiv preprint arXiv:1703.01488*, 2017. URL https://arxiv.org/abs/1703.01488. 2

[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*, 2017. URL https://arxiv.org/abs/1706.03762. 4, 5

[45] Jianwei Wang, Guoyang Shen, Yanhong Wu, Haoran Li, Hao Peng, Huiping Zhuang, Cen Chen, and Ziqian Zeng. FreoStream: Enhancing Stream Guardrails via Future-Aware Reasoning and Safety-Aligned Optimization. *arXiv preprint arXiv:2606.13737*, 2026. URL https://arxiv.org/abs/2606.13737. 3

[46] Zhengxuan Wu, Aryaman Arora, Atticus Geiger, Zheng Wang, Jing Huang, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders. *arXiv preprint arXiv:2501.17148*, 2025. URL https://arxiv.org/abs/2501.17148. 4

[47] Jiaqi Yu, Xin Wang, Yixu Wang, Jie Li, Yan Teng, Xingjun Ma, and Yingchun Wang. SentGuard: Sentence-Level Streaming Guardrails for Large Language Models. *arXiv preprint arXiv:2606.02041*, 2026. URL https://arxiv.org/abs/2606.02041. 2, 3

[48] M. Yuan and Y. Lin. Model selection and estimation in regression with grouped variables. JRSS-B 68(1):49-67, 2006. URL https://doi.org/10.1111/j.1467-9868.2005.00532.x. 8

[49] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. ShieldGemma: Generative AI Content Moderation Based on Gemma. *arXiv preprint arXiv:2407.21772*, 2024. URL https://arxiv.org/abs/2407.21772. 2, 3

[50] Haiquan Zhao, Chenhan Yuan, Fei Huang, Xiaomeng Hu, Yichang Zhang, An Yang, Bowen Yu, Dayiheng Liu, Jingren Zhou, Junyang Lin, Baosong Yang, Chen Cheng, Jialong Tang, Jiandong Jiang, Jianwei Zhang, et al. Qwen3Guard Technical Report. *arXiv preprint arXiv:2510.14276*, 2025. URL https://arxiv.org/abs/2510.14276. 2, 3

[51] Carolina Zheng, Nicolas Beltran-Velez, Sweta Karlekar, Claudia Shi, Achille Nazaret, Asif Mallik, Amir Feder, and David M. Blei. Model Directions, Not Words: Mechanistic Topic Models Using Sparse Autoencoders. *arXiv preprint arXiv:2507.23220*, 2025. URL https://arxiv.org/abs/2507.23220. 3

[52] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, et al. Representation Engineering: A Top-Down Approach to AI Transparency. *arXiv preprint arXiv:2310.01405*, 2023. URL https://arxiv.org/abs/2310.01405. 3

Appendix

A Additional prefill results

Per-topic overlap. Enterprise documents and HR & people operations are the two hardest topics for nearly every detector on Gemma 3 4B (Table 7). Both are procedural, internal-process topics with shared vocabulary, since a memo about a staffing policy and an HR plan read alike. A stronger detector also narrows the spread between topics, from 9 overlap points for the summation GRU to 3.5 for the dynamic ConvNeXt.

Supervising prompt tokens. Every detector in the main text is trained with labels on both prompt and response tokens. Adding the roughly 515,000 prompt tokens to the 8.2M response tokens leaves response scores nearly unchanged (Table 8). Ten of the twelve differences are within 1.2 overlap points, and the two larger ones are both static on Gemma 3 4B, where GRU gains 2.4 points and ConvNeXt loses 2.1. GRU likely gains because its running state now receives gradient during the prompt and reaches the response already oriented.

	Summation			Dynamic		
Topic	GRU	ConvNeXt	Transformer	GRU	ConvNeXt	Transformer
Enterprise documents	72.1	83.9	80.9	79.4	93.4	84.5
General news & content	**81.3**	**90.8**	84.9	**90.4**	**96.4**	89.5
Customer service	79.5	86.4	86.0	86.0	93.4	89.0
Legal	79.3	83.1	**87.7**	87.2	95.7	**91.1**
Financial	78.1	86.9	85.4	85.4	93.9	88.6
HR & people ops.	74.1	84.7	82.2	83.5	92.9	86.8
Healthcare	79.6	85.8	**87.7**	89.9	95.6	**91.1**

Table 7: Per-topic test overlap (%) on Gemma 3 4B.

Input	GRU	ConvNeXt	Transformer
Summation, Gemma 3 1B	92.2 → 92.5	94.6 → 94.3	92.3 → 91.1
Summation, Gemma 3 4B	79.8 → 82.2	91.0 → 88.9	88.4 → 87.2
Dynamic, Gemma 3 1B	94.5 → 94.8	95.8 → 96.0	90.0 → 89.8
Dynamic, Gemma 3 4B	91.5 → 90.4	96.2 → 95.7	89.6 → 90.3

Table 8: Overlap (%) on response tokens, response-only supervision → prompt-and-response supervision.

Scores pooled over prompt and response tokens run 1 to 3 points below response-only scores, because short prompt questions give the detector less context than long answers. All main-text scores are computed on the pooled set.

B Layer knockout and feature attribution

Accuracy alone does not show which layers or features a detector relies on, and a detector that relies on a single layer is exposed to any shift in that layer's signal. We measure this dependence with two methods, applied to every trained detector. In layer knockout, we zero one layer's block of input features, re-score the detector on the test split, and repeat for each of the ten layers. A layer's knockout drop is the overlap lost when it is zeroed. The largest drop gives the detector's worst case, and the sum of the ten drops shows how much it depends on individual layers overall.

In attribution, we compute gradient-times-input [40] for the correct-topic logit at each response token, which measures how much each input feature contributed to the correct-topic logit. Within the layer with the largest knockout drop, we then zero the ten highest-ranked and, separately, the ten lowest-ranked features. Finally, we check whether the two methods agree on which layers matter, using the rank correlation between each layer's share of the attribution and its knockout drop.

The worst-layer knockout applies to every input, including NoSAE. The summed knockout and the attribution tests compare how the static rules choose features, so they cover only summation, F-statistic and group lasso.

The selection rule changes how much a detector depends on any one layer. Zeroing layer 29 alone reduces the overlap of the Gemma 3 4B summation GRU by 32.5 points, whereas under the F-statistic its most important layer accounts for only 1.2. Dynamic features and NoSAE spread the dependence most evenly, with no detector losing more than 4.7 points to one layer apart from the Gemma 3 1B NoSAE Transformer (11.4).

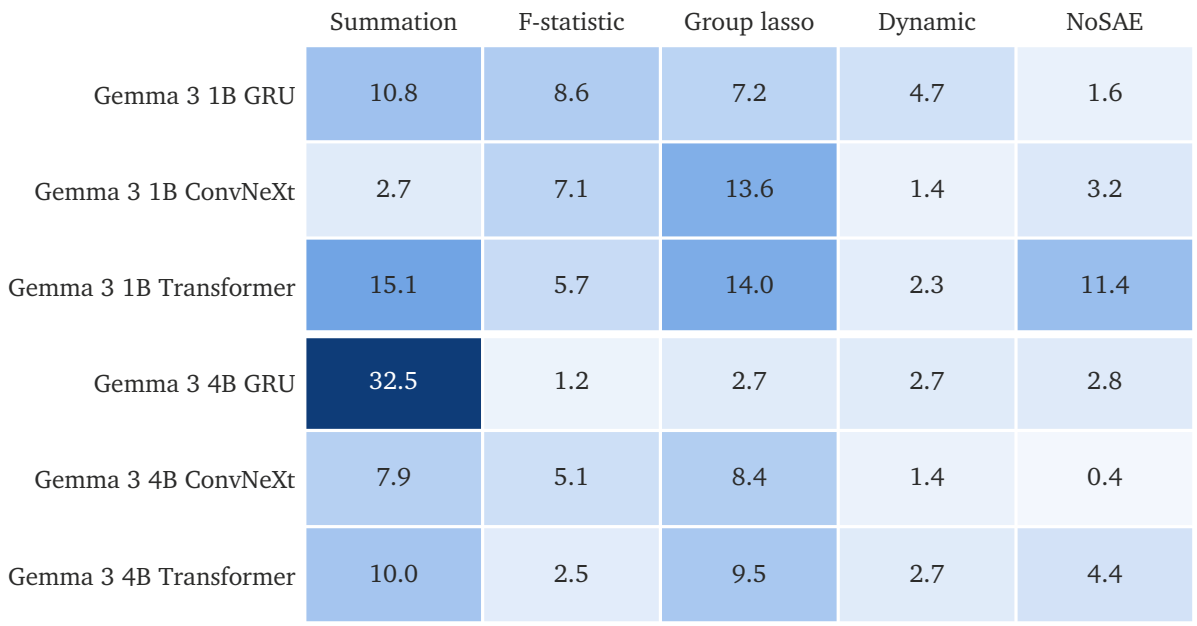


Figure 8: Overlap lost (percentage points) when the single most important layer is zeroed.

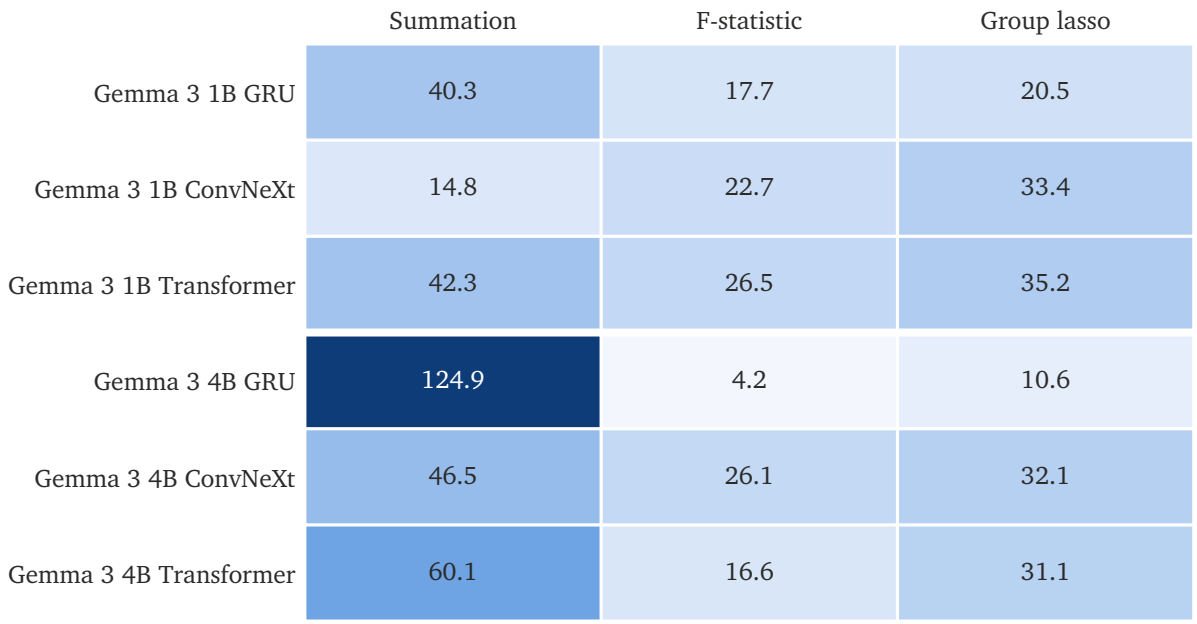


Figure 9: Overlap lost (percentage points), summed over the ten single-layer knockouts.

Label-based selection lowers the summed knockout for five of the six detectors, most sharply for the Gemma 3 4B GRU (124.9 to 4.2). The exception is the Gemma 3 1B ConvNeXt, whose total rises to 33.4 under group lasso, almost all of it on layer 16, while its score rises slightly too. The rule also changes which layers a detector draws on. Summation relies mostly on late layers, while group lasso on Gemma 3 1B moves all three detectors onto layer 16, the earliest of the ten.

Within a layer, attribution identifies the features a detector uses. Zeroing the ten highest-attributed features of the most important layer costs 0.1 to 10.3 overlap points, against at most 0.16 for the ten lowest, and the top ten cost more in all 18 detector-and-rule combinations. Under group lasso, zeroing the 75 lowest-attributed features costs at most 0.8 points, so about half of each layer's budget contributes little. For dynamic features the attribution is spread thinly, with the top 100 of about 54,000 columns holding 6 to 16% of the total.

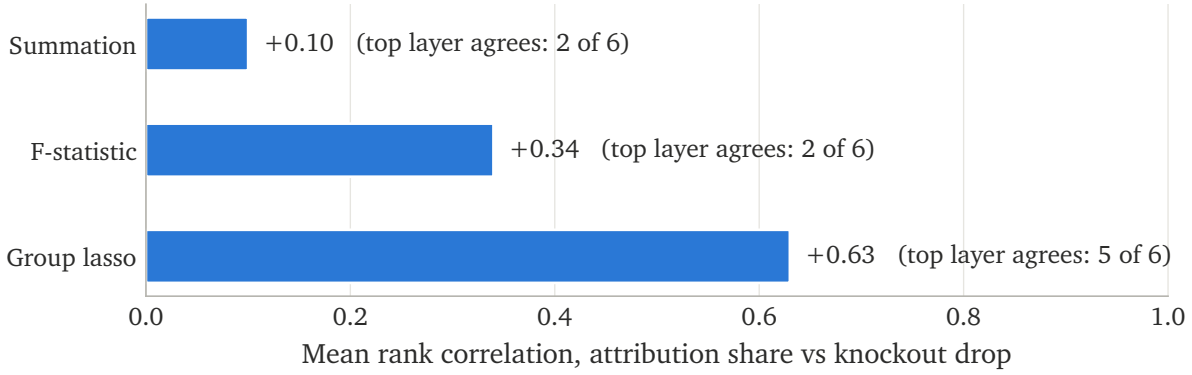


Figure 10: Agreement between attribution share and knockout drop across layers.

Across layers, attribution is less dependable (Figure 10). Attribution sums absolute contributions, so a layer with large activations receives a large share whether or not the detector depends on it. Summation picks features by the size of their activations, so it inflates exactly those layers. We therefore rank layers by knockout.

C Detector architectures and training details

All three detectors read the same per-token input and emit logits over the seven topics at every position. Every detector projects its input to width 256 and ends in a linear head over the seven topics, and all three share the training settings in Table 9. Each is trained with token-level cross-entropy over the labelled tokens, and the checkpoint with the best validation macro-F1 is kept. The body of each detector is the same across inputs, so the parameter counts differ only through the input projection (Table 6 in the main text). The input width itself is set by each backbone's feature-extraction settings, listed in Table 10.

Hyperparameter	GRU	ConvNeXt	Transformer
Layers / blocks	3	6	2
Hidden width	256	256	256
Feed-forward width	–	1,024	512
Attention heads	–	–	8
Kernel size	–	7	–
Dilations	–	1–32	–
Context (tokens)	all	379	all
Layer scale	–	0.01	–
Positional encoding	–	–	none
Learning rate	3×10^{-4}	1×10^{-3}	3×10^{-4}
Optimiser	AdamW	AdamW	AdamW
Weight decay	0.01	0.01	0.01
Batch size	16	16	16
Epochs	15	15	15
Dropout	0.1	0.1	0.1
Gradient clipping	1.0	1.0	1.0
Seed	42	42	42

Table 9: Detector hyperparameters.

Setting	Gemma 3 1B	Gemma 3 4B
Layers read	16–25	24–33
Hidden width	1,152	2,560
SAE	Gemma Scope 2	Gemma Scope 2
SAE variant	`l0_small`	`l0_small`
SAE features per layer	16,384	16,384
Activation	JumpReLU	JumpReLU
K per layer	150	150
Static width (Summation, F-statistic)	1,500	1,500
Static width (Group lasso)	1,438	1,489
Dynamic columns per layer (max)	6,000	6,000
Dynamic width	∼54,000	∼54,000
Dynamic non-zero per token	∼1,380	∼1,380

Table 10: Feature extraction settings.